

多代理 AI 協作制度規格書

版本 1.2 | 日期 2026-06-13 | 性質 可複製的制度設計,源自真實運轉中的四代理系統 **v1.2 新增:**角色界線升級為代碼閘門(含風險分級緊急通道)、車道邊界納入設計階段、雙向自治閉環(有界 worker + 遞迴護欄 + 演習啟用)、教訓擴充至九則

1. 這套制度解決什麼問題

單一 AI 助手再強,都有四個結構性缺陷:

- 沒人審查它**——AI 自己寫的東西自己驗收,錯誤會安靜地累積。
- 上下文有限**——長專案跨越數十次對話,每次都從失憶開始。
- 沒有治理的記憶**——要嘛什麼都不記,要嘛什麼都記(包括錯的)。
- 人類變成郵差**——多個 AI 工具之間的訊息傳遞全靠人肉複製貼上。

本制度用「**角色分離 + 檔案協調 + 審查閘門 + 治理記憶 + 排程自動化**」五個支柱解決以上問題。整套制度在一台普通的 Windows 個人電腦上運轉,不需要伺服器,不需要資料庫,核心只是一個資料夾和幾條紀律。

2. 三條公理

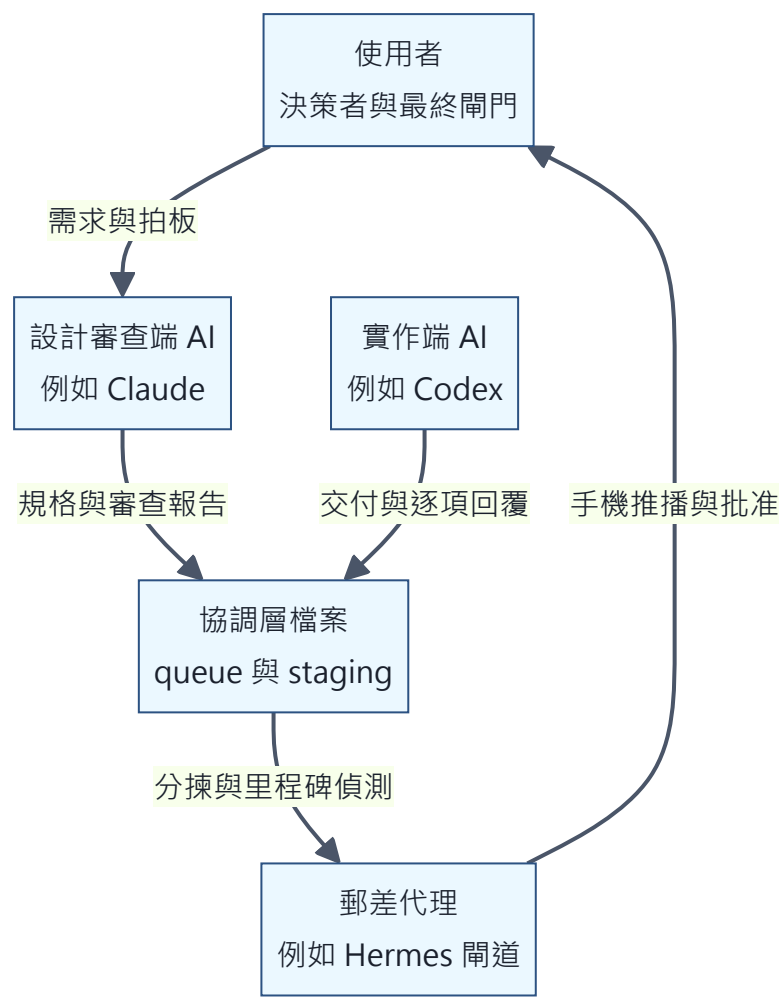
整套制度建立在三條公理上,其他一切都是推論:

公理一:球員不兼裁判。 設計與審查的 AI 不寫生產程式碼;實作的 AI 不驗收自己的交付。審查者自己寫的程式碼,沒有人能客觀審查——角色分離是品質迴圈成立的前提。

公理二:檔案是唯一真相。 所有協調走純文字檔案(append-only),不依賴任何一方的對話記憶。「現在進度如何」的答案永遠來自檔案,不來自誰的印象。

公理三:閘門必須寫進代碼,只寫在文件不算。 「需要批准才能執行」如果只是文件裡的一句話,而排程器跑著上膛的代碼——那批准就已經被繞過了。每一道閘門都要有對應的機制(旗標、檔案檢查、權限)強制執行。

3. 角色架構



角色	職責	明確禁止
使用者	提需求、決策、批准高風險閘門	不需要當訊息中繼站
設計審查端	寫規格、根因診斷、獨立審查交付	不下場改生產程式碼
實作端	依規格實作、附證據交付、逐項回覆	不自審、不自行標記通過
郵差代理	通知推播、進件分揀、手機批准介面	不直接寫入正式知識庫

角色綁的是「職責」不是「品牌」——任何兩個以上能讀寫檔案的 AI 工具都能套用。

4. 協調層:檔案就是訊息匯流排

```

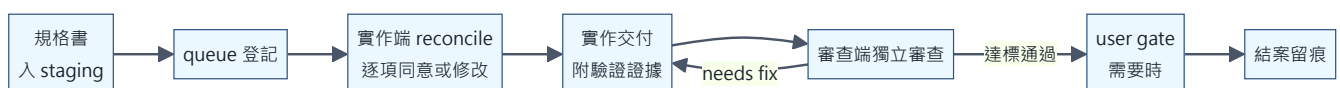
coordination/
├─ queue.md           ← 任務登記簿(append-only,唯一真相)
├─ broadcast.md       ← 廣播訊息(append-only)
├─ README.md         ← queue 條目格式契約 + 認領契約
├─ staging/
│   └─ <任務ID>/      ← 每個任務一個資料夾
│       ├── spec 或提案
│       ├── reconcile(實作端逐項回覆)
│       ├── 交付報告與證據
│       ├── 審查報告
│       └─ coordination_claim.json(認領章)
│   └─ ...
└─ outbox/           ← 持久化回軌(供自動執行器認領)

```

核心慣例:

- **任務 ID:** YYYYMMDD-簡短名稱, 全程引用同一 ID。
- **append-only:** queue 不改舊條目, 狀態變化追加新 section; 讀取端以同一任務的最新 section 為準(教訓四)。
- **格式契約:** 條目欄位的寫法明文規定, 所有自動解析器必須相容人手寫的寬鬆格式——我們曾因解析器只認舊格式, 導致整個自動化對新任務「全盲」三天而無人察覺。
- **認領機制(claim/lease):** 任何代理處理某任務資料夾前, 先寫入認領章(誰、何時、租約到期); 其他代理看到有效認領即退避, 過期租約可重佔。多代理並行時, 「兩個人拿同一封信」從此是機制性不可能, 不靠默契。

任務生命週期



reconcile 是關鍵一步: 實作端對規格逐項回覆 agree / modify / reject, 有異議就地協商。架構級變更必須雙方同意才動工——避免「規格一廂情願、實作另起爐灶」。

5. 審查制度

審查的第一原則: 不信報告, 重跑驗證。

審查端收到交付後的標準動作:

1. 讀 diff 與交付報告;
2. **獨立重跑**所有驗證(編譯、測試、驗證腳本)——不採信對方貼的結果;
3. 補跑對方漏列的驗證項;

4. 對可疑行為寫探針實測重現;
5. 依固定 rubric 評分 + 列出 required / non-blocking 項;
6. 審查報告入 staging, 結論入 queue。

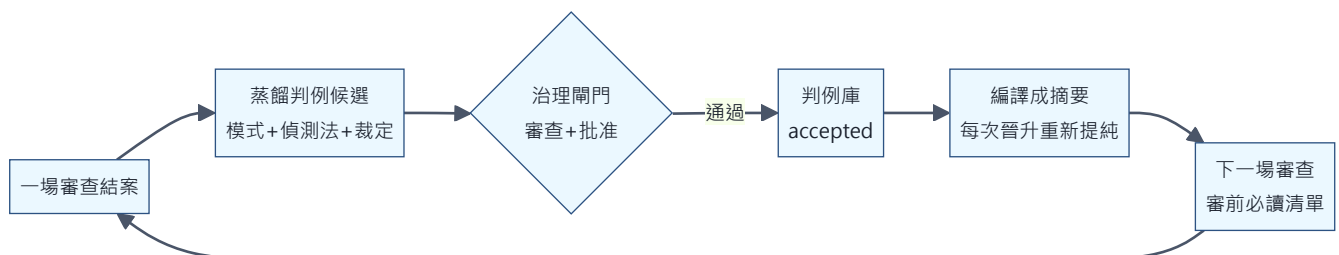
固定 rubric(防裁判漂移):四維度各 25 分——規格符合度、證據品質、安全與閘門紀律、運行耐久性;80 分放行線。每條 finding 帶兩個標籤:嚴重度(HIGH/MEDIUM/LOW/INFO)與**確定性**(PROVEN=審查者重跑或直讀證實/INFERRED=讀碼推論/SPECULATIVE=合理懷疑無直接證據)。鐵律:**SPECULATIVE 永不單獨擋件**;硬檢查(測試/編譯/稽核腳本)的結果永遠優先於模型判斷。

迭代有上限:needs_fix 迴圈最多三輪,第三輪仍不過即停,升級給使用者或退回設計——不准無限乒乓。修正輪只准修列出的必修項,禁止「順手重構」。

雙層審查:排程器喚起的無人值守初審(限唯讀)+人在場時的互動深審。**權威順序固定:**硬檢查 > 互動工具級審查 > 自動受限審查——低層永不推翻高層。自動審查看不到它需要的證據時,必須回報「證據缺口,升級」,而不是 needs_fix 或放行(教訓五)。

6. 審查庭:讓審查隨案例變強

審查制度管「這一次審得對」;審查庭管「下一次審得更好」。核心是**判例制度**:



- **判例:**重大審查裁定蒸餾成獨立條目——失敗模式、怎麼偵測、裁定為何、適用邊界。一次性 bug 不立判例,會跨任務復發的模式才立。
- **審前必讀:**所有審查者(包括自動審查)開審前載入已通過的判例清單當檢查表。
- **判例也走治理:**候選 → 審查 → 批准,和其他共享記憶同一條生命週期;過時判例走廢止,不靠自然遺忘。
- **學術對應:**這是把 Reflexion(記憶作為跨 session 的外層改進迴圈)制度化——**系統的審查能力隨著每一場審查單調遞增,且不依賴任何一個對話的記憶存活。**

我們的判例庫開張首日就自我應驗:第五條判例「為教訓寫的修復自己也是未測路徑」墨跡未乾,當天就抓到一個只在「從未執行過的刷新路徑」上爆炸的 bug。

7. 閘門制度

並非所有事都讓 AI 自動做。風險分級:

風險級	例子	處理
低	文件、測試、內部腳本	實作端自動執行,事後審查
中	行為變更、排程任務、通知管道	審查通過後執行
高	花錢、憑證、對外發布、刪除資料	user gate: 使用者明文批准才動

批准必須結構化留痕。口頭「好啊」之後,記錄成機器可查的欄位:

```
user_approved_<事項>: <ISO時間戳> | <批准人> | <範圍>
原話: 「好,就照方案 A 執行」
```

這讓三個月後的任何人(或任何 AI)都能回答「這件事誰、何時、批准了什麼範圍」。

公理三的實戰版:我們曾發生「文件寫了要等批准、但排程器已在跑新代碼」——系統自己把第一槍開了。教訓固化為制度:**宣告閘門的同時,必須指出強制它的機制在哪一行代碼或哪個設定**,審查時逐一核對。協調層契約現在明文寫著:「文件裡的一句話是對閘門的請求,不是閘門本身。」

7.1 車道閘門:把角色界線變成代碼,不是叮嚀

公理一(球員不兼裁判)如果只寫在文件裡,總會有一個代理判斷錯、把不該自己寫的產品碼下海——失敗模式從來不是「規則沒寫」,是「沒認出這次該歸誰」。所以把它做成**寫前閘門**:設計/審查代理嘗試編輯實作端的產品碼(或協調基礎設施)時,在動作發生前被擋下,回一句「這是對方車道,寫規格交付」。文件能被忽略,寫前 deny 不能。

但**不能寫死**——重大事故時必須能即時處理,不能等對方代理醒來。所以緊急通道由**風險門檻代碼驗證**,而非絕對封鎖:

- 預設:擋。
- 放行條件:**高風險,或中風險但會直接導致系統卡死/構成資安風險**;且附新鮮時間戳(例如 60 分鐘內);且每次破例自動寫入稽核日誌。
- 低風險、例行工作、過期或不合格的破例:仍然擋。

這就是「閘門在代碼、例外要刻意且過得了風險門檻、破例必留案底」——不是寫死,是「預設交對方、緊急秒繞、繞過必驗風險」。

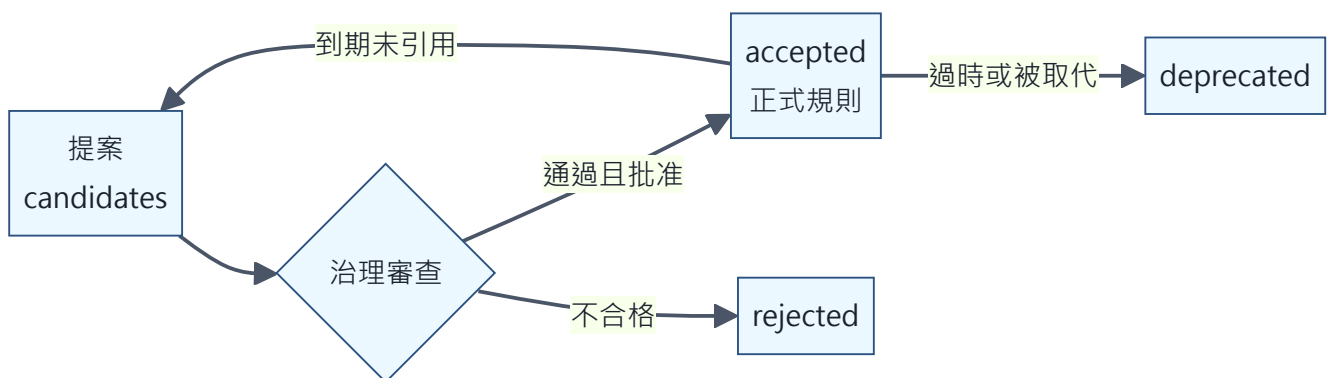
7.2 車道邊界是設計的一部分,不是事後補

每當評估或設計一個新專案,就要當場判斷它會不會產出「對方車道」的產品碼。會的話:**先定義風險**(哪個路徑是產品碼;產品碼 vs 協調/規格/探針的邊界;為何直接編輯有險),**再把該路徑註冊進閘門清單**——在實作開始前完成,而非事後補。新產品庫預設絕不留未保護。

8. 記憶治理(記憶宮殿)

跨代理的長期記憶不是日誌堆,是需要治理的共享資產。

```
memory/  
├─ shared/  
│   ├── candidates/ ← 提案區(任何代理可寫)  
│   ├── accepted/   ← 正式規則(僅閘門可晉升)  
│   ├── rejected/   ← 不合格提案(留檔可查)  
│   ├── deprecated/ ← 過時規則  
│   └─ _maintenance/ ← 維護腳本與週報  
└─ private/<代理>/ ← 各代理私有區(互不可讀)
```

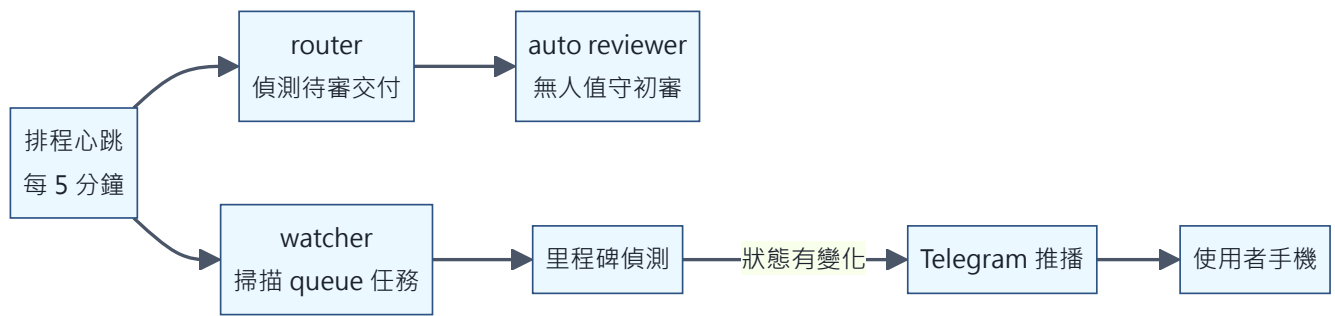


硬規則(違反即拒收):

- 不收秘密(token、密碼、.env);
- 不收原始聊天記錄與 session 日誌——記憶收的是「策展後的規則」,不是流水帳;
- 不收業務領域知識(那屬於知識庫,不屬於代理記憶);
- 每筆記憶必帶審計欄位(誰、何時、為何變更)+ append-only 變更歷史;
- 晉升 accepted 需通過審查閘門,任何代理不得自提自批。

自動老化: candidates 超過 30 天未審→提醒; accepted 設到期日,到期未被引用→退回重審。每週排程維護產出健康報告(pending 老化、到期倒數、重複 ID、秘密掃描)。

9. 自動化管線:讓郵差失業



一個作業系統排程任務,每 5 分鐘跑一輪純腳本管線:

- **watcher** 解析 queue,分揀出「待實作 / 待回覆 / 待批准」的任務並標優先級;
- **router** 偵測 staging 裡出現的新交付,自動產生審查請求(無主要工件時標 wait-for-artifact,不空轉);
- **auto reviewer** 喚起唯讀 AI 完成初審(先認領,審完釋放);
- **里程碑偵測** 對「回覆落地 / 審查出爐 / 等待批准」等事件,推播一行訊息到使用者手機,寫明任務名與下一棒是誰。

設計鐵則:通知失敗不影響管線(best-effort);自動化只分揀與通知,生命週期變更(晉升、結案)永遠過關門;每 5 分鐘的心跳本身是零 LLM 成本(純本地腳本,模型只在真的有審查請求時出場)。

9.1 閉環:有界的自主派工

上面的管線讓「通知」自動化;最後一哩是讓「動工」也自動化——系統在偵測到待辦時自己喚醒對應代理,雙向皆然(實作端接規格、審查端接交付),人完全不在迴圈裡傳話。關鍵設計:

- **事件驅動,不是排班**:那個零成本的偵測器本來就知道何時有活——在偵測到的當下才喚起一次 worker。沒活時零成本,有活時數分鐘內動工;比任何固定週期都即時,又不空轉燒錢。
- **worker 嚴格有界**(自主寫檔安全的命脈):寫入只准它自己的任務資料夾、shell 只准驗證指令(編譯/跑測試)、無網路、隱形啟動、單飛、每日上限——全部由代碼閘門強制,不是提示詞約束。
- **遞迴護欄(不可妥協)**:自主代理絕不可對「它自己的派工/審查機制」的變更自動動工——那類任務一律升級給人類監督的層級。能自我批准自己審批機制的系統,等於沒有閘門。
- **啟用本身是最高閘門**:這個能力只在實彈演習證明它真的有界後才開——演習要同時證明(a)正常派工跑通、(b)各種越界嘗試全被擋死、(c)自指涉任務正確拒絕自動跑——再加上人類明確批准。

做到這一步,「人類當郵差」才真正結束:人只在被要求決策與批准時出現,傳話、查狀態、催進度、轉交付全部消失。

10. Token 經濟:三層階梯

個人預算不是公司無限額度——但**省錢的前提是品質不降**。三層階梯,只有最上層是目標:

層級	含義	制度態度
剋扣	餵更少、更破碎的輸入來省錢	禁止 。實證:被餵截斷證據的審查連發三次錯誤判決,加上覆核收拾,總花費比一次吃飽的審查更貴——剋扣是雙輸
壓縮	同樣判斷力,更少 token	允許的省錢手段,但有底線 :用濃縮材料(例如判例摘要)做出的判斷,必須與讀完整原文一致——等價驗證不通過,濃縮版就不得使用
提純	同樣 token,更高判斷力	方向 。摘要每次更新都重新蒸餾——跨案合成、磨利偵測線索、汰除冗餘,每個 token 承載的判斷力隨積累上升

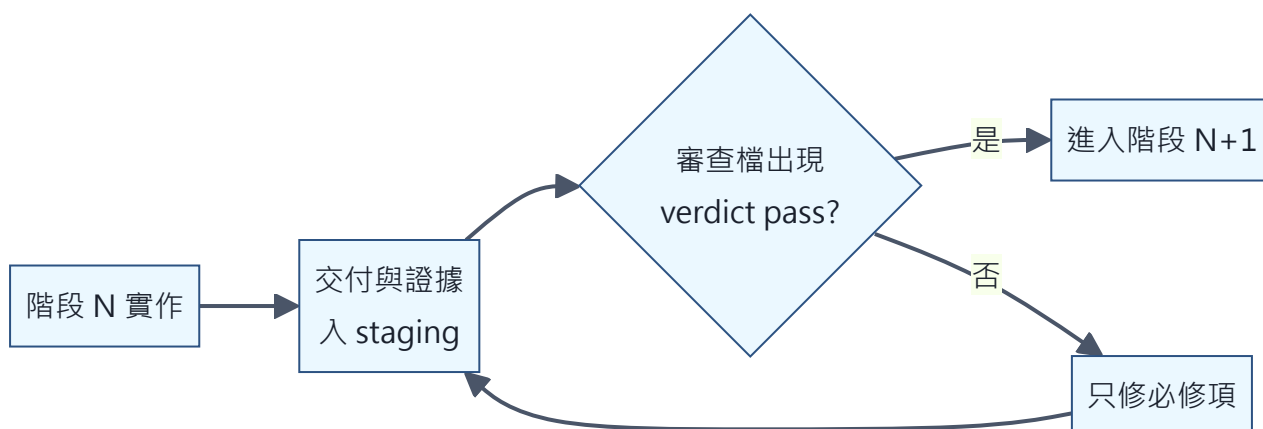
地板是「品質不降」,天花板不存在——制度只獎勵往上。

操作化原則:

- **決定性優先**: compile/測試/稽核腳本是零 token 且比模型更可靠——紅燈直接結案,模型一滴不沾;
- **不重複勞動**: 交付物雜湊未變就引用前次判定,不重審;
- **折疊呼叫**: 新的抽取需求折疊進既有 LLM 呼叫的結構化輸出,不另開常駐呼叫;
- **新增常駐 LLM 步驟必附週耗估算**——靜默的經常性燒錢視同高危缺陷(預算跟閘門一樣,要寫在代碼跑的地方);
- **衝突時品質贏**: 省錢與品質衝突的個案,品質優先,成本記帳留待重新設計——永不靠降級審查默默吸收。

11. 長程目標模式:階段閘門

對於跨多日的大目標,使用實作端的長程模式(如 Codex /goal)讓它持續推進,但**自治不等於免審**:



- 目標定義寫成檔案(目標、可驗證的完成條件、約束、各階段範圍),goal 指令指向該檔;
- **階段閘門是檔案狀態**: staging 出現審查檔且 verdict 為 pass,才准進下一階段——機制化,不靠自覺;互動審查與自動審查衝突時,互動優先;

- 卡關超過 24 小時:更新 queue 狀態並暫停,不准跳關;
- 最終完成條件是一支稽核腳本,逐項輸出 proven / gated / missing,全 proven 才算達標。

實戰紀錄:這套閘門曾在無人值守的深夜通過極限測試——實作端 AI 收到一份「通用審查通過」文件,自行認定它不滿足階段專屬閘門而拒絕通關;嘗試自行喚起審查被自身安全政策擋下,它沒有繞過,而是登記正式委派、寫下完整診斷、原地暫停到天亮。閘門的價值不在順境,在沒人看著的時候。

12. 實戰教訓九則

每一條都是真實付過學費的:

#	教訓	事故摘要
1	解析器要寬容,寫入要規範	queue 格式自然演化出兩種方言,自動化只認舊方言,對新任務全盲多日——而健康報告天天顯示「一切正常」
2	「從未發生」要驗證,不能推定	通知功能上線數日,翻遍歷史發現它在生產環境一次都沒真正發送過,連測試都是 dry-run
3	閘門寫進代碼,文件不算	文件宣告「待批准」,排程器卻已在跑上膛的新代碼,自己開了第一槍
4	append-only 需要「最新狀態為準」的讀法	任務多次追加狀態,掃描器只看第一段,把早已結案的任務當成待辦殭屍
5	證據缺口不是缺陷	自動審查因看不到證據連續誤判不通過;正確裁定是「升級給看得到的審查層」,不是對內容判死
6	多線並行先認信封	多個任務同時進行時,代理拿錯任務資料夾的風險真實存在;解法是任務 ID 紀律 + 認領機制
7	改審查工具的人正是被審的人時,必須標註	實作端為了解鎖閘門修了審查器本身——方向是收緊所以無害,但「自指涉變更」必須成為交付報告的強制標註類別,靠運氣抓不是制度
8	為教訓寫的修復,自己也是未測路徑	修「掃描器失明」的補丁上線次日,它自己的「同日刷新」分支第一次被執行就崩潰——每條新路徑都要有踩過它的測試
9	不信報告,連自己的檢查也不信	驗收時用了一個有 bug 的檢查指令(無論有無命中都回報「有問題」),差點誤報一個不存在的越界;重驗才發現是自己的檢查寫錯。硬檢查本身也要驗它對不對

13. 最小可行導入(給想自己搭一套的人)

不需要一次蓋完。四步成長:

第一步(十分鐘):建檔案協調層。 一個 `queue.md`、一個 `staging/` 資料夾、一頁格式契約。從此兩個 AI 之間的交接全部過檔案,人只負責「叫對方去看信」。

第二步(半天):立審查紀律。 角色分離、reconcile 逐項回覆、審查獨立重跑、固定 rubric 與 80 分放行線、needs_fix 三輪上限。此時品質迴圈已經成立,自動化還一行都不用寫。

第三步(漸進):逐件自動化。 排程心跳 → watcher 分揀 → 推播通知 → 自動初審 → 認領機制 → 長程 goal 模式。每加一件,用第十二節的教訓對照檢查一次。

第四步(持續):開審查庭。 重大審查結案就蒸餾判例,判例過閘門進記憶,審前必讀。從這一步起,系統不只是在運轉——它在變強。鍛造不是一次成形:每輪新討論都該把舊結論拿回火上重審,多層摺疊淬火,才有大馬士革鋼的強度。

成長的判斷標準只有一個:人類在流程裡做的事,是不是只剩「決策與批准」。 其他一切——傳話、查狀態、催進度、轉交付——都應該逐步消失。

附錄:術語對照

術語	含義
reconcile	實作端對規格的逐項回覆(agree / modify / reject)
staging	每個任務專屬的工件資料夾
user gate	必須由使用者明文批准的檢查點
needs_fix	審查不通過,附必修清單(最多三輪)
claim/lease	處理前先認領,租約到期可重佔,防止重複處理
里程碑事件	值得通知人類的狀態變化
記憶宮殿	治理式共享記憶(candidates 到 accepted 生命週期)
判例	從審查裁定蒸餾的可復用模式(偵測法+裁定+邊界)
確定性標籤	finding 的證據等級:PROVEN / INFERRED / SPECULATIVE
再蒸餾	摘要在每次晉升時重新提純,判斷密度隨積累上升
proven / gated / missing	稽核腳本對完成條件的三態判定
車道閘門	寫前攔截,擋住代理編輯不屬於它車道的程式碼;附風險分級緊急通道
風險分級破例	緊急通道的放行門檻:高風險,或中風險 + 系統卡死/資安;低風險/過期仍擋
遞迴護欄	自主代理不得對自己的派工/審查機制變更自動動工,一律升級人類層

本規格書源自一套真實運轉的系統:兩個 AI 編碼代理、一個訊息闡道代理與一位使用者,在一台 Windows 個人電腦上,用純文字檔案完成了從需求到交付的完整理。文中所有教訓均為實際事故的覆盤。